

PROGRAM 5:-

AIM

To solve the **Missionaries and Cannibals Problem** using a state-space search technique by safely transporting all missionaries and cannibals from one side of the river to the other without violating the given constraints.

ALGORITHM

1. Start the program.
2. Represent the state as (M, C, B), where M is the number of missionaries, C is the number of cannibals, and B represents the boat position.
3. Initially, all missionaries and cannibals are on the left side of the river.
4. Generate all possible boat movements carrying 1 or 2 people.
5. Check whether the new state is valid:
 - Missionaries should never be outnumbered by cannibals when missionaries are present.
 - The number of missionaries and cannibals must remain within the valid range.
6. Avoid states that have already been visited.
7. Continue generating valid states until all missionaries and cannibals reach the opposite side.
8. Display the sequence of states representing the solution.
9. Stop the program.

PROGRAM

```
from collections import deque

def is_safe(m_left, c_left, m_total, c_total):
    m_right = m_total - m_left
    c_right = c_total - c_left
    if (m_left > 0 and c_left > m_left):
        return False
    if (m_right > 0 and c_right > m_right):
        return False
    return True

def solve(m_total, c_total):
```

```

start = (m_total, c_total, 'L')
goal = (0, 0, 'R')
queue = deque([(start, [start])])
visited = set()

while queue:
    (m, c, boat), path = queue.popleft()
    if (m, c, boat) == goal:
        print("\nSolution:")
        for state in path:
            print(state)
        return
    if (m, c, boat) in visited:
        continue
    visited.add((m, c, boat))
    moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]
    for dm, dc in moves:
        if boat == 'L':
            nm, nc = m - dm, c - dc
            new_boat = 'R'
        else:
            nm, nc = m + dm, c + dc
            new_boat = 'L'
        if (0 <= nm <= m_total and
            0 <= nc <= c_total and
            is_safe(nm, nc, m_total, c_total)):
            queue.append(((nm, nc, new_boat), path + [(nm, nc, new_boat)]))

print("No solution found.")

m = int(input("Enter number of Missionaries: "))
c = int(input("Enter number of Cannibals: "))

```

solve(m, c)

OUTPUT

```
Enter number of Missionaries: 5
Enter number of Cannibals: 2
```

Solution:

```
(5, 2, 'L')
(3, 2, 'R')
(4, 2, 'L')
(2, 2, 'R')
(3, 2, 'L')
(3, 0, 'R')
(3, 1, 'L')
(1, 1, 'R')
(2, 1, 'L')
(0, 1, 'R')
(1, 1, 'L')
(0, 0, 'R')
```

RESULT

The **Missionaries and Cannibals Problem** was **successfully solved** by applying state-space search and finding a sequence of valid boat movements that safely transfers all missionaries and cannibals to the opposite side.